

Curso

Herramientas de Computación en la Nube

Clase 7
Febrero 14 de 2026

Codificación de borrado

RustFS

Anexo Técnico

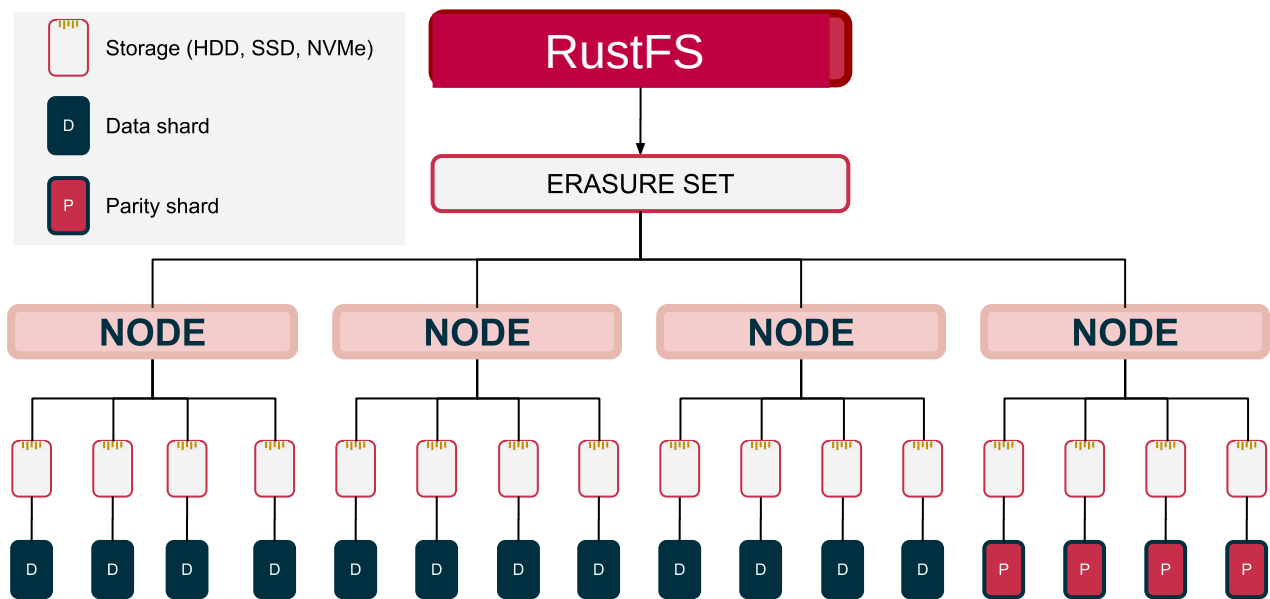
Este documento se fundamenta en l documentación oficial de Rust en
<https://docs.rustfs.com/concepts/principle/erasure-coding.html>

Profesor
Alvaro Mauricio Montenegro Díaz
Universidad de la Sabana
Febrero 2026

1. Qué es codificación de borrado (erasure code)

El Código Reed-Solomon (Código RS) es un código de borrado basado en estructuras algebraicas de campos finitos. Gracias a su eficiente capacidad de recuperación de datos y su configuración de redundancia flexible, se utiliza ampliamente en diversos campos. A continuación, detallamos sus principales escenarios de aplicación desde dos dimensiones: campos técnicos y aplicaciones prácticas.

En vez de guardar réplicas completas del objeto, se guarda el objeto fragmentado + paridad, de modo que si pierdes algunos fragmentos, reconstruyes el objeto usando álgebra.



2. Parámetros k, m, n y qué significan de verdad

- k: número de *fragmentos de datos* (data shards).
- m: número de *fragmentos de paridad* (parity shards).
- $n = k + m$: total de fragmentos que se distribuyen en discos/nodos.

Propiedad clave:

- Con un esquema RS(k,m), puedes perder hasta m fragmentos (por fallas de discos/nodos) y recuperar el objeto, siempre que aún tengas cualquier conjunto de k fragmentos disponibles.

Ejemplo RS(10,4)

- 10 datos + 4 paridad = 14 fragmentos.
- Tolera pérdida simultánea de 4 fragmentos.
- Overhead de paridad: $m/k = 4/10 = 40\%$ (más eficiente que 3 réplicas, que es 200% extra).

3. Por qué RS es tan usado

Reed–Solomon es un código MDS (Maximum Distance Separable):

- Para una tolerancia de fallas dada (m), RS usa la mínima redundancia posible para que “cualquier k reconstruya”.

En almacenamiento, eso se traduce en:

- Menos costo en capacidad que replicación.
- Gran durabilidad (si se configura y opera bien).
- Trade-off: reconstrucción consume CPU + red (repair / rebuild).

4. Qué ocurre en un sistema tipo RustFS (visión de pipeline)

Escritura (PUT)

1. El cliente sube un objeto.
2. RustFS lo divide en k trozos (shards) de tamaño fijo (p.ej., 1–4 MB).
3. Calcula m shards de paridad.
4. Distribuye esos n shards en distintos “targets” (discos/nodos).
5. Guarda metadatos: dónde quedaron shards, tamaño, hash, etc.

Lectura (GET)

1. Pide el objeto.
2. RustFS intenta leer shards.
3. Si encuentra $\geq k$ shards válidos $\rightarrow$ reconstruye/ensambla y responde.
4. Si detecta shards faltantes $\rightarrow$ puede disparar repair (recrear shards y reescribirlos).

Esto es exactamente lo que refleja un diagrama

Coordinator $\rightarrow$ *Nodes* $\rightarrow$ *Decoder* $\rightarrow$ *Repairer*

5. Núcleo matemático: por qué con k shards basta

RS trabaja sobre un campo finito $GF(2^w)$. En sistemas de almacenamiento suele ser $GF(2^8)$ o $GF(2^{16})$, porque:

- El dato se procesa en bytes/words.

- Las operaciones (suma y multiplicación) son cerradas y eficientes con tablas/SIMD.

Idea clave

Se modela el “bloque” como un vector D de k elementos:

$$D = [d_1, \dots, d_k]$$

y se genera un vector C de n elementos (k datos + m paridad) usando una matriz generadora (G):

$$C = D \cdot G$$

En práctica:

- Las primeras k filas de G suelen ser identidad $\rightarrow$ los primeros k shards “son” el dato.
- Las m filas restantes mezclan linealmente los datos $\rightarrow$ paridad.

Por qué recuperas con cualquier k

Porque RS elige (G) de forma que cualquier submatriz $k \times k$ que corresponda a k shards disponibles sea invertible. Entonces:

$$D = C_{\{k\}} \cdot G_{\{k\}}^{-1}$$

donde ($C_{\{k\}}$) son los k shards que sí tienes.

6. Vandermonde, interpolación

Otra forma equivalente (útil para entender) es interpolación:

- Construyes un polinomio ($p(x)$) de grado $(k-1)$ que “codifica” los datos.
- Evaluas ese polinomio en n puntos distintos $\rightarrow$ obtienes n valores = shards.
- Con cualquier k puntos, puedes reconstruir el polinomio (interpolación) y recuperar los datos.

Esto explica por qué RS es tan robusto: es como “muestras” suficientes de una función en un campo finito.

7. Recuperación/repair: el coste real (CPU + red)

7.1. Cuando faltan shards:

- Hay que resolver un sistema lineal para reconstruir los perdidos.
- Implementaciones modernas optimizan:
 - Eliminación gaussiana sobre $GF(2^w)$
 - Jerasure / ISA-L (Intel) o kernels SIMD
 - Multiplicación de matrices con tablas log/antilog o instrucciones vectoriales

En la práctica, el tiempo de recuperación depende de:

- tamaño de objeto
- ancho de banda de red entre nodos
- throughput de discos
- capacidad SIMD/CPU

7.2. Si el sistema repara “en línea” o “en background”

“Ajuste dinámico (k,m)”

En sistemas reales, el matiz es:

- Puedes tener políticas distintas por bucket/tier (hot/warm/cold) y elegir RS(k,m) distinto.
- Pero cambiar (k,m) para datos ya escritos implica reescribir (re-encode) objetos existentes (operación pesada).
- Lo “dinámico” suele ser: *para nuevos objetos o por tiering*, no magia instantánea.

8. Qué es la matriz generadora (G)

En un código RS(k, m):

- Tienes un vector de datos D con k elementos.
- Quieres producir ($n = k + m$) fragmentos codificados.
- Eso se logra multiplicando:

$$C = D \cdot G$$

donde:

- (D) es un vector ($1 \times k$)
- (G) es una matriz ($k \times n$)
- (C) es un vector ($1 \times n$)

8.1. Estructura típica de (G)

En almacenamiento distribuido (como RustFS), se usa una forma sistemática:

$$G = \begin{bmatrix} I_k & P \end{bmatrix}$$

donde:

- I_k = matriz identidad ($k \times k$)
- P = matriz de paridad ($m \times k$)

Esto produce:

$C = \begin{bmatrix} \text{datos originales} \\ \text{paridad} \end{bmatrix}$

8.2. Ejemplo simple: RS(2,2)

($k=2, m=2$)

Matriz generadora:

$G = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 1 \\ 1 & 2 \end{bmatrix}$

Los dos primeros renglones son identidad $\rightarrow$ datos directos. Los dos últimos generan paridad.

8.3. Cómo se construye realmente (G)

En RS clásico se usa una matriz de Vandermonde.

Si elegimos elementos distintos ($\alpha_1, \dots, \alpha_n$) en el campo finito:

$$G_{ij} = \alpha_j^i$$

En práctica:

- Se eligen puntos distintos del campo $GF(2^8)$
- Se construye una matriz que garantiza que cualquier submatriz ($k \times k$) sea invertible

Eso es clave.

8.4. Propiedad fundamental: submatrices invertibles

La condición crítica de RS es:

Cualquier subconjunto de k columnas de G debe formar una matriz invertible.

¿Por qué?

Porque si pierdes hasta m fragmentos, te quedan k fragmentos.

Para recuperar:

$$D = C_{\{k\}} \cdot G_{\{k\}}^{-1}$$

Si esa submatriz no fuera invertible $\rightarrow$ no podrías recuperar.

Reed–Solomon garantiza esta propiedad (es MDS).

8.5. Interpretación geométrica

Piensa que:

- Los datos son coeficientes de un polinomio de grado $(k-1)$.
- La matriz generadora evalúa ese polinomio en distintos puntos.
- Cada fragmento es una evaluación.
- Con k evaluaciones puedes reconstruir el polinomio completo.

Eso es interpolación de Lagrange sobre un campo finito.

8.6. Qué ocurre computacionalmente

En almacenamiento real:

- Las operaciones no son sobre números normales.
- Son sobre $GF(2^8)$ (bytes).
- Las sumas son XOR.
- Las multiplicaciones usan tablas log/antilog o SIMD.

Entonces cada shard de paridad es:

$$p_j = \sum_{i=1}^k g_{ji} \cdot d_i$$

donde (g_{ji}) pertenece al campo finito.

8.7. Por qué se usa forma sistemática

Si no fuera sistemática:

- Los datos originales también estarían mezclados.
- Toda lectura requeriría decodificación.

Con forma sistemática:

- Si no hay fallas $\rightarrow$ lees directamente.
- Solo decodificas cuando hay pérdida.

Eso mejora latencia.

8.8. Relación con rendimiento en RustFS

En implementación práctica:

- (G) se pre-calcula.
- Se almacena como tabla.
- Se usan multiplicaciones vectorizadas (AVX2 / NEON en ARM).
- Se hace “matrix multiply” sobre bloques de bytes.

La eficiencia depende directamente de:

- tamaño del shard
- optimización SIMD
- paralelismo (Rayon, threads)
- cache locality

9. Resumen conceptual

La matriz generadora (G) :

- Define cómo se mezclan los datos.
- Garantiza tolerancia a fallos máxima.
- Hace posible la reconstrucción con cualquier k fragmentos.
- Es el núcleo algebraico del sistema.

Sin (G) , no hay erasure coding.

10. Diccionario técnico

Almacenamiento Distribuido y Codificación de Borrado

AVX / AVX2 (Advanced Vector Extensions)

Extensiones del conjunto de instrucciones de CPU diseñadas para ejecutar operaciones vectoriales sobre múltiples datos en paralelo (SIMD).

AVX2 permite operaciones enteras vectorizadas de 256 bits.

En codificación Reed–Solomon, se utiliza para acelerar multiplicaciones en campos finitos y operaciones de matrices.

B-Tree (Balanced Tree)

Estructura de datos balanceada utilizada para indexación eficiente en almacenamiento persistente. Garantiza complejidad $O(\log n)$ en búsqueda, inserción y eliminación. Frecuentemente empleada en motores relacionales como PostgreSQL o InnoDB para gestión de metadatos.

Blake3

Algoritmo criptográfico moderno de hashing.
Optimizado para paralelismo y alto rendimiento.
Utilizado para generar sumas de comprobación (checksums) en fragmentos de datos (shards).

Bucket

Unidad lógica de organización en almacenamiento tipo S3.
Contenedor de objetos dentro de un namespace global.
Equivalente conceptual a un “directorio raíz”, aunque internamente es plano.

Campo Finito (Finite Field)

Estructura algebraica con número finito de elementos donde las operaciones suma, resta, multiplicación y división (excepto por cero) están definidas y cerradas.
Reed–Solomon opera típicamente sobre $GF(2^8)$ o $GF(2^{16})$.

CRC (Cyclic Redundancy Check)

Código de detección de errores basado en división polinómica modular.
Permite detectar corrupción accidental de datos, pero no corregirla.

Durabilidad (Durability)

Probabilidad estadística de que los datos permanezcan intactos a lo largo del tiempo.
En sistemas EC depende de:

- k y m
- distribución física
- probabilidad de falla de nodos

EC (Erasure Coding)

Técnica de redundancia basada en fragmentación de datos y generación de paridad algebraica.
Permite recuperar información perdida sin replicar el objeto completo.

FFT (Fast Fourier Transform)

Algoritmo eficiente para evaluación e interpolación de polinomios.
Algunas implementaciones de Reed–Solomon utilizan variantes FFT en campos finitos para acelerar codificación.

$GF(2^8)$ (Galois Field)

Campo finito con 256 elementos.

Cada elemento representa un byte.

Las multiplicaciones se realizan módulo un polinomio irreducible (ej. 0x11D).

Hash

Función determinística que transforma datos arbitrarios en un valor fijo de tamaño reducido.

Se utiliza para:

- Integridad
- Verificación
- Detección de corrupción

Interpolación de Lagrange

Método matemático para reconstruir un polinomio de grado $k-1$ a partir de k puntos.

Base teórica del proceso de recuperación en Reed–Solomon.

KMS (Key Management Service)

Servicio de gestión de claves criptográficas.

Permite cifrado en reposo mediante claves controladas externamente o localmente.

Latencia

Tiempo transcurrido entre la solicitud y el inicio de la respuesta.

En almacenamiento distribuido se ve afectada por:

- red
- reconstrucción EC
- carga del sistema
-

LSM Tree (Log Structured Merge Tree)

Estructura de almacenamiento optimizada para escritura secuencial.

Utilizada en bases de datos orientadas a escritura intensiva.

MDS (Maximum Distance Separable)

Clase de códigos que alcanza la máxima tolerancia a fallos para un nivel de redundancia dado.

Reed–Solomon es un código MDS.

Matriz Generadora (G)

Matriz algebraica utilizada para transformar el vector de datos en vector codificado.

En forma sistemática:

$G =$

$\begin{bmatrix}$

I_k

\P
\end{bmatrix}

Garantiza que cualquier subconjunto de k fragmentos sea suficiente para reconstrucción.

Metadata

Información descriptiva asociada a un objeto (tamaño, ubicación de shards, checksums, timestamps).

Nodo (Node)

Unidad física o lógica de almacenamiento dentro de un cluster distribuido.

NVMe (Non-Volatile Memory Express)

Interfaz de almacenamiento de alta velocidad para SSDs modernos.

Proporciona baja latencia y alto throughput.

Objeto (Object)

Unidad básica de almacenamiento en sistemas tipo S3.

Compuesto por:

- datos
- metadatos
- identificador único (key)

Overhead

Cantidad adicional de almacenamiento requerido para redundancia.

En RS(k,m):

$$\text{Overhead} = \frac{m}{k}$$

Paridad (Parity Shard)

Fragmento calculado algebraicamente a partir de los datos originales.

Permite reconstrucción ante pérdida.

PUT / GET / DELETE

Operaciones básicas del protocolo S3:

- PUT: escritura
- GET: lectura
- DELETE: eliminación

RAID (Redundant Array of Independent Disks)

Tecnología tradicional de redundancia a nivel de servidor.

Diferente de EC distribuido a nivel cluster.

Reed–Solomon (RS)

Código de borrado algebraico basado en evaluación de polinomios sobre campos finitos.
Permite recuperación con cualquier conjunto de k fragmentos.

Repair / Rebuild

Proceso de reconstrucción automática de fragmentos faltantes y redistribución en el cluster.

Shard

Fragmento individual de datos o paridad generado por EC.

S3 (Simple Storage Service)

Protocolo/API de almacenamiento de objetos definido originalmente por AWS.
Estándar de facto para object storage.

SIMD (Single Instruction Multiple Data)

Modelo de paralelismo donde una instrucción opera simultáneamente sobre múltiples datos.
Fundamental para optimizar codificación RS.

SSD (Solid State Drive)

Dispositivo de almacenamiento basado en memoria flash.
Más rápido y con menor latencia que HDD.

Throughput

Tasa de transferencia de datos (MB/s, GB/s).
Afecta tiempos de codificación y reconstrucción.

TLS (Transport Layer Security)

Protocolo criptográfico para comunicación segura.
Protege datos en tránsito.

Vandermonde Matrix

Tipo de matriz utilizada en construcción de códigos RS.
Garantiza invertibilidad de submatrices.

Vectorización

Uso de registros vectoriales del CPU para acelerar procesamiento paralelo.